

## 설치 비용의 차이와 모델의 차이점이 주된 차이점

Hv6m가까이 6Y각각의경계각각의W61공근거리  
Hm공근s\_h\_Wa공근(6가공근4공>459??9AA4: 89A  
H 0kpd15

k kpd 6 hpd 4k pd

( 6 ) 6

Ha 낮기공 6 4k pd

[ ~~W~~ N 문 구                      동 문 구 ]

hpd                      H

LSH는 무작위 함수를 사용:

- 데이터 분포 미고려
- 실제 이웃 관계 무시
- 높은 거짓양성율

문제:

벡터 A와 B가 유사하지만  
LSH 함수의 무작위성으로 인해  
다른 해시 버킷에 들어갈 수 있음

k pd                      H

### 데이터의 실제 이웃 구조를 분석:

1. 데이터로부터 이웃 그래프 구성
2. 이웃 관계를 유지하는 해시 함수 설계
3. 유사한 점들이 같은 해시 버킷에 들어가도록 학습

결과: LSH보다 정확하고 효율적인 검색

## [ VWS

gkk      H

각 점  $p$ 에 대해:

1. 모든 다른 점과의 거리 계산
2. 가장 가까운 k개 선택
3.  $p \rightarrow \{k\text{개 이웃}\}$  연결

예 ( $k=3$ ):

점 A: [B, C, D] (A와 가장 가까운 3개)  
점 B: [A, E, F]  
점 C: [A, D, G]  
...

H

방향 그래프:

- A가 B의 이웃이라고 해서 B가 A의 이웃은 아닐 수 있음

상호 이웃 (mutual neighbors):

- A와 B가 서로를 이웃으로 포함하는 경우
- 더 강한 유사도 신호

H

KNN 그래프 구성:  $O(n^2)$   
(모든 점 쌍의 거리 계산)

대규모 데이터: 근사 KNN 그래프 사용

- 샤할링 또는 클러스터링으로 근사
- $O(n \log n)$  또는  $O(n\sqrt{n})$

[WAV 동문구

H

목표: 해시 함수  $h$ 를 설계하여

- 이웃인 점들은 같은 해시 값 확률 높음
- 다른 점들은 다른 해시 값 확률 높음

제약:

- 해시 값은  $0 \sim m-1$  범위 ( $m$ 개 버킷)
- 계산 비용 낮아야 함

H

최대화:

$\sum_{\{(i,j) \in \text{edges}\}} [h(i) = h(j)]$   
(이웃 쌍이 같은 버킷)

최소화:

$\sum_{\{(i,j) \notin \text{edges}\}} [h(i) = h(j)]$   
(비이웃 쌍이 같은 버킷)

[WAV 동문구

9H과갈구행각간결거01

그래프 라플라시안 계산:

$L = D - A$   
(D: 차수 행렬, A: 인접 행렬)

라플라시안의 고유벡터 사용:

$v_1, v_2, \dots$  (가장 작은 고유값부터)

해시 함수:

$h(x) = [\text{sign}(v_1^T x), \text{sign}(v_2^T x), \dots]$   
(이진 해시 코드 생성)

결과: 각 비트가 이웃 보존성 최적화

: H 0 결5갈단

목표: 이웃을 최대한 같은 그룹으로  
비이웃을 다른 그룹으로

알고리즘:

1. 그래프에서 최소 컷 찾기
2. 이웃은 같은 쪽, 비이웃은 다른 쪽
3. 각 컷이 하나의 비트를 정의

결과: 이웃 관계를 명시적으로 보존하는 해시

>H

마르코프 연쇄 몬테카를로(MCMC) 사용:

1. 현재 해시 할당 상태에서 시작

- 2. 점진적으로 할당 개선
- 3. 수렴 시 최적 또는 근최적 해시

장점: 이론적 보장 가능  
단점: 계산 비용 높음

[W9]

계 H

함수 1:  $h_1(x) = [bit\_0\_1, bit\_1\_1, \dots]$   
함수 2:  $h_2(x) = [bit\_0\_2, bit\_1\_2, \dots]$   
...  
함수 k:  $h_k(x) = [bit\_0\_k, bit\_1\_k, \dots]$

각 함수는 이웃 구조의 다른 측면 캡처

H

```
procedure NSH_SEARCH(query_q, K, k_tables):
    all_candidates = []

    for hash_function_idx = 1 to k_tables:
        h_idx = hash_function_idx

        // 쿼리의 해시 코드 계산
        query_hash = compute_hash(q, h_idx)

        // 같은 해시 코드의 모든 점 찾기
        bucket_contents = hashtable[h_idx][query_hash]
        all_candidates.extend(bucket_contents)

    // 중복 제거
    all_candidates = unique(all_candidates)

    // 거리 기반 정렬
    candidates_with_dist = [
        (c, distance(q, c)) for c in all_candidates
    ]

    // 가장 가까운 K개 반환
    return sorted(candidates_with_dist,
                  key=lambda x: x[1])[:K]
```

[W10]

H

이웃 보존율 (Recall):  
= (검색된\_이웃\_개수) / (실제\_이웃\_개수)

정밀도 (Precision):  
= (실제\_이웃) / (검색된\_점\_개수)

NSH의 이론:  
적절한 k\_tables 선택 시  
높은 recall과 precision 달성 가능

H

검색:  $O(k\_tables \times \text{평균\_버킷\_크기} + \text{정렬})$

대부분의 경우:  
 $O(k\_tables \times (\text{평균\_버킷\_크기}))$   
평균 버킷 크기  $\ll n$

[W11] 동문구 합필동문구 합필동문사

|  |     |     |      |
|--|-----|-----|------|
|  | hpd | kpd | dkpt |
|--|-----|-----|------|

~~~~~

|  |  |  |      |
|--|--|--|------|
|  |  |  | k 7V |
|  |  |  |      |
|  |  |  |      |
|  |  |  |      |
|  |  |  |      |
|  |  |  |      |

[ W0e

kpdH

문제:

1. 필터를 만족하는 점들의 부분집합으로 이웃 그래프 구성

2. 해시 함수는 필터 만족 점들 사이의 이웃 보존

알고리즘:

// 단계 1: 필터된 KNN 그래프 구성

filtered\_points = {p | filter\_predicate(p)}

knn\_graph = build\_knn\_graph(filtered\_points, k)

// 단계 2: NSH 함수 학습

hash\_functions = learn\_nsh\_functions(knn\_graph)

// 단계 3: 검색

query\_candidates = search\_with\_hash\_functions(q)

return filter\_results\_by\_distance(

query\_candidates, K

)

H

필터 선택도 높음 (많은 점 선택):

- 필터 없는 NSH와 유사

- 이웃 구조 더 풍부

- 좋은 성능

필터 선택도 낮음 (적은 점 선택):

- 이웃 구조 희소

- 해시 함수 학습 어려움

- 성능 저하 가능

[ W0g

H

정적 데이터: 한 번만 구성

동적 데이터: 주기적 재구성 필요

- 시간 비용 높음 ( $O(n^2)$ )

- 근사 기법 필요

H

k개 해시 함수 저장:

- 스펙트럼 해싱: k개 벡터 (고유벡터)

- 비트맵: 각 함수당 n비트

- 메모리:  $O(k \times n)$  비트

H

이웃 쌍들이 자주 함께 접근:

→ 캐시 미스 감소

→ 메모리 대역폭 효율성 향상

→ 전체 검색 속도 15~30% 향상

[ W0NL 동문구

H

새로운 점 x 추가:

- 1. 기존 이웃 그래프에서 x의 k개 이웃 찾기
- 2. 기존 점들의 이웃도 잠재적으로 변경
- 3. 영향받는 해시 함수 부분 재학습

전체 재구성보다 빠름:

시간:  $O(k \times n)$  vs  $O(n^2)$

[ WNN

5 H

벡터 공간의 이웃 관계와  
필터 조건의 상관성 분석:

예:

"category=전자제품" 필터  
→ 전자제품 벡터들이 서로 이웃일 확률

만약 높은 상관성:

- 필터 조건별 별도 NSH 인덱스 구성
- 각 인덱스가 더 효율적

낮은 상관성:

- 필터와 무관하게 전체 NSH 사용
- 필터링은 별도로 수행

[ WNS 동문구 3 4

데이터: GIST, SIFT 벡터들 (백만 개)

회수율 (Recall):

NSH: 97-99%  
LSH: 85-90%  
HNSW: 98-99%

쿼리당 시간:

NSH: 2-5ms  
LSH: 3-8ms  
HNSW: 0.5-2ms

메모리:

NSH: 2GB  
LSH: 2.5GB  
HNSW: 3GB

|     |      |   |   |        |        |        |       |   |     |
|-----|------|---|---|--------|--------|--------|-------|---|-----|
| 96  |      | H |   |        |        |        |       |   | P   |
| : 6 | kpdH |   | 4 |        | kpd    |        |       |   | P   |
| >6  |      | H | 4 |        |        |        | P     |   |     |
| ?6  | Hkpd |   |   | 4      |        |        |       |   | P   |
| A6  |      |   | H | 0      | 14     |        |       |   | kpd |
|     | P    |   |   |        |        |        |       |   |     |
| B6  |      | H | 0 | H같은 공간 | N같은 공간 | N같은 공간 | 같은 공간 |   |     |
|     | P    |   |   |        |        |        |       |   |     |
| C6  | H    |   | 4 |        | kpd    |        |       | P |     |
| D6  |      | H |   |        | kpd    |        | P     |   |     |